Matplotlib Interview Questions — Answers & Hints

Complete study guide based on the uploaded TOPS Technologies interview-preparation sheet • 46 Questions

How to use this guide: Try answering each question yourself first, then check the hint, answer, and code example. For interview preparation, focus on explaining the concept in your own words rather than memorizing code.

1. What is data visualization, and why is it important in data analytics?

Hint: Think about how visuals help make sense of large or complex data.

Answer: Data visualization is the graphical representation of data using charts, graphs, maps, and other visual elements. It is important because it makes patterns, trends, comparisons, outliers, and relationships easier to understand than raw tables of numbers. In data analytics, good visualization helps analysts communicate findings and supports faster, better decisions.

2. When would you choose to use Matplotlib over Seaborn for creating a chart?

Hint: Focus on flexibility, customization, and level of control.

Answer: Choose Matplotlib when you need fine-grained control over almost every visual element, want highly customized charts, need unusual chart layouts, or are working close to the underlying plotting objects. Seaborn is often more convenient for statistical visualizations and attractive defaults, while Seaborn itself is built on Matplotlib.

3. Explain the difference between a Figure, Axes, and Axis in Matplotlib.

Hint: Each term refers to a different part of the plotting structure.

Answer: A Figure is the overall canvas or window containing one or more plots. An Axes is an individual plotting area where data is drawn; a Figure can contain multiple Axes. An Axis is the x-axis or y-axis object associated with an Axes and controls things such as ticks, labels, and limits.

4. Describe the basic workflow for creating a plot using Matplotlib.

Hint: Think from importing the library to displaying or saving the result.

Answer: A typical workflow is: import Matplotlib, prepare the data, create a Figure/Axes or use pyplot, plot the data, customize the title/labels/legend/grid, then display or save the figure.

Example:

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]
y = [10, 20, 15, 25]
plt.plot(x, y)
plt.title('Sales Trend')
plt.xlabel('Month')
plt.ylabel('Sales')
plt.show()
```

5. How do you create a simple line chart in Matplotlib using plt.plot() ?

Hint: Pass x and y values to plt.plot().

Answer: Use plt.plot(x, y) to connect data points with a line.

Example:

```
plt.plot(months, sales)
plt.show()
```

6. Which parameters of plt.plot() would you adjust to change the color, marker style, and line width of a line?

Hint: Look for color, marker, and linewidth arguments.

Answer: Use color (or c) for line color, marker for point style, and linewidth (or lw) for line thickness.

Example:

```
plt.plot(x, y, color='red', marker='o', linewidth=2)
```

7. How would you add a title and axis labels to a Matplotlib chart?

Hint: There are specific functions to set these elements.

Answer: Use plt.title() for the chart title, plt.xlabel() for the x-axis label, and plt.ylabel() for the y-axis label.

Example:

```
plt.title('Monthly Sales')
plt.xlabel('Month')
plt.ylabel('Revenue')
```

8. Write a code snippet to plot monthly sales data as a line chart, with circle markers and a red line.

Hint: Combine plt.plot() with marker and color.

Answer: Use the marker='o' argument for circle markers and color='red' for the line.

Example:

```
import matplotlib.pyplot as plt

months = ['Jan', 'Feb', 'Mar', 'Apr']
sales = [120, 150, 135, 180]

plt.plot(months, sales, color='red', marker='o')
plt.title('Monthly Sales')
plt.xlabel('Month')
plt.ylabel('Sales')
plt.show()
```

9. What is the difference between a vertical bar chart and a horizontal bar chart in Matplotlib?

Hint: Focus on the direction of the bars.

Answer: A vertical bar chart uses plt.bar(), with categories along the x-axis and values extending vertically. A horizontal bar chart uses plt.barh(), with categories on the y-axis and values extending horizontally. Horizontal bars are especially useful when category names are long.

10. How do you create a horizontal bar chart using Matplotlib?

Hint: There is a specific function for horizontal bars.

Answer: Use plt.barh(categories, values).

Example:

```
categories = ['North', 'South', 'East', 'West']
revenue = [400, 350, 300, 450]
plt.barh(categories, revenue)
plt.show()
```

11. What parameter would you use to change the width of the bars in a bar chart?

Hint: Think of the argument that controls bar thickness.

Answer: Use the width parameter in plt.bar(). For a horizontal bar chart, use height in plt.barh().

Example:

```
plt.bar(categories, values, width=0.6)
```

12. How can you add text labels to each bar in a bar chart?

Hint: You can place text at each bar's position.

Answer: You can loop through the bars and use plt.text(), or use Axes.bar_label() when working with bar containers. bar_label() is convenient for displaying the bar values.

Example:

```
bars = plt.bar(categories, values)
plt.bar_label(bars)
plt.show()
```

13. Write a code snippet to plot regional revenue data as a vertical bar chart, and add value labels on top of each bar.

Hint: Use plt.bar() and label each bar.

Answer: Create the bars with plt.bar() and add labels using bar_label().

Example:

```
import matplotlib.pyplot as plt

regions = ['North', 'South', 'East', 'West']
revenue = [450, 380, 520, 610]

bars = plt.bar(regions, revenue)
plt.bar_label(bars)
plt.title('Regional Revenue')
plt.xlabel('Region')
plt.ylabel('Revenue')
plt.show()
```

14. What is the purpose of a pie chart, and when is it most appropriate to use one?

Hint: Think about showing parts of a whole.

Answer: A pie chart shows how categories contribute to a whole, usually as percentages or proportions. It is most appropriate for a small number of categories where the part-to-whole relationship is the main message. Too many slices make a pie chart difficult to read.

15. How do you display percentage values on each segment in a pie chart using Matplotlib?

Hint: Look for an argument in plt.pie().

Answer: Use the autopct parameter, such as autopct='%1.1f%%', to display percentage labels.

Example:

```
plt.pie(values, labels=labels, autopct='%1.1f%%')
plt.show()
```

16. How can you highlight or 'explode' one segment of a pie chart?

Hint: Use the explode parameter.

Answer: Pass a sequence to explode, where non-zero values move the corresponding slice away from the center.

Example:

```
plt.pie(values, labels=labels, explode=[0, 0.1, 0, 0], autopct='%1.1f%%')
```

17. Describe how to create a donut chart in Matplotlib.

Hint: Think about a pie chart with a hole in the center.

Answer: Create a pie chart and use the wedgeprops parameter with a width value smaller than the full radius. The unused center becomes the hole, producing a donut chart.

Example:

```
plt.pie(values, labels=labels, wedgeprops={'width': 0.4})
plt.show()
```

18. Write a code snippet to visualize market share data as a pie chart with exploded largest segment and percentage labels.

Hint: Find the largest value and give its slice a non-zero explode value.

Answer: Identify the largest market-share value, create an explode list with a small offset for that index, and use autopct for percentages.

Example:

```
import matplotlib.pyplot as plt

companies = ['A', 'B', 'C', 'D']
share = [35, 25, 20, 20]
explode = [0.08, 0, 0, 0]

plt.pie(share, labels=companies, explode=explode, autopct='%1.1f%%')
plt.title('Market Share')
plt.show()
```

19. What does a histogram show, and how is it different from a bar chart?

Hint: Think about data distribution vs categorical values.

Answer: A histogram shows the distribution of a numeric variable by grouping values into continuous intervals called bins. A bar chart compares separate categories. Histogram bars represent numeric ranges, while bar-chart bars represent categories.

20. How do you control the number of bins in a histogram in Matplotlib?

Hint: Look for the bins argument.

Answer: Use the bins parameter in plt.hist(). For example, bins=10 creates 10 bins.

Example:

```
plt.hist(ages, bins=10)
plt.show()
```

21. Explain the difference between plotting density and frequency in a histogram.

Hint: Think about counts versus a normalized distribution.

Answer: Frequency shows the number of observations in each bin. Density normalizes the histogram so the total area is approximately 1, allowing distributions to be compared even when sample sizes differ. In Matplotlib, density=True produces a density histogram.

22. Write code to plot a histogram of customer ages with 10 bins.

Hint: Use plt.hist() with bins=10.

Answer: Pass the age values to plt.hist() and set bins=10.

Example:

```
import matplotlib.pyplot as plt

ages = [21, 25, 29, 31, 35, 36, 42, 45, 48, 52, 55]
plt.hist(ages, bins=10)
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.title('Customer Age Distribution')
plt.show()
```

23. What kind of data relationship does a scatter plot show?

Hint: Think about how two numeric variables relate.

Answer: A scatter plot shows the relationship between two numerical variables. It can reveal positive or negative association, clusters, outliers, and possible nonlinear patterns. It does not by itself prove causation.

24. How can you add a trend line to a scatter plot in Matplotlib?

Hint: Consider fitting a line using numpy.

Answer: Fit a regression line, commonly with NumPy's polyfit(), then evaluate it with polyval() and plot the fitted values over the scatter plot.

Example:

```
import numpy as np

m, b = np.polyfit(x, y, 1)
plt.scatter(x, y)
plt.plot(x, m*x + b)
plt.show()
```

25. Write code to plot advertising spend vs sales as a scatter plot and add a regression line.

Hint: Use plt.scatter() and NumPy's polyfit().

Answer: Create the scatter plot, fit a first-degree polynomial, then plot the fitted line.

Example:

```
import numpy as np
import matplotlib.pyplot as plt

ad_spend = np.array([10, 20, 30, 40, 50])
sales = np.array([25, 35, 43, 55, 65])

plt.scatter(ad_spend, sales)
m, b = np.polyfit(ad_spend, sales, 1)
plt.plot(ad_spend, m * ad_spend + b)
plt.xlabel('Advertising Spend')
plt.ylabel('Sales')
plt.show()
```

26. What is the difference between plt.subplot() and plt.subplots() ?

Hint: One returns axes objects, the other sets up the grid.

Answer: plt.subplot() selects or creates one subplot using a grid specification and is convenient for quick layouts. plt.subplots() creates a Figure and one or more Axes objects together and is generally cleaner for programmatic work and multiple plots.

27. How can you create a 2x2 grid of plots with shared x-axis using Matplotlib?

Hint: Look for parameters in plt.subplots().

Answer: Use plt.subplots(2, 2, sharex=True). This creates four Axes arranged in two rows and two columns with a shared x-axis.

Example:

```
fig, ax = plt.subplots(2, 2, sharex=True)
plt.show()
```

28. Write code to display four different charts in a single figure using subplots.

Hint: Create a 2x2 grid and plot on each Axes.

Answer: Create four Axes with plt.subplots(2, 2), then call the relevant plotting method on each Axes.

Example:

```
fig, ax = plt.subplots(2, 2, figsize=(10, 7))
ax[0, 0].plot(x, y)
ax[0, 1].bar(categories, values)
ax[1, 0].hist(ages, bins=10)
ax[1, 1].scatter(x, y)
plt.tight_layout()
plt.show()
```

29. How do you change the overall figure size in a Matplotlib plot?

Hint: Think about figsize.

Answer: Use figsize=(width, height), usually in inches, when creating the Figure.

Example:

```
fig, ax = plt.subplots(figsize=(10, 6))
```

30. What is the purpose of plt.style.use() in Matplotlib?

Hint: Think about themes and appearance.

Answer: plt.style.use() applies a predefined style sheet to change the overall appearance of plots, including elements such as gridlines, fonts, and default visual settings.

Example:

```
plt.style.use('ggplot')
```

31. How can you customize the color palette of your charts in Matplotlib?

Hint: Think about passing colors explicitly or using colormaps.

Answer: You can specify colors directly with arguments such as color, use a list of colors for multiple series, or select a Matplotlib colormap when a continuous or categorical palette is appropriate.

32. Write code to plot a sales trend chart using the 'ggplot' style and a custom color.

Hint: Apply the style before plotting and pass a color to plt.plot().

Answer: Apply the ggplot style, then create the line chart with your chosen color.

Example:

```
import matplotlib.pyplot as plt

plt.style.use('ggplot')
plt.plot(months, sales, color='purple')
plt.title('Sales Trend')
plt.show()
```

33. Explain what a stacked bar chart is and when you might use one.

Hint: Think about showing totals and their component categories.

Answer: A stacked bar chart places multiple series on top of one another within each category. It is useful when you want to compare category totals while also showing how each total is composed of subcategories.

34. How can you add error bars to a line or bar chart in Matplotlib?

Hint: Look for the yerr/xerr parameters.

Answer: Use yerr for vertical uncertainty and xerr for horizontal uncertainty. For example, plt.errorbar() is designed for error-bar plots, while bar charts can accept yerr.

Example:

```
plt.errorbar(x, y, yerr=errors, fmt='o-')
```

35. Describe how to plot a time series chart in Matplotlib.

Hint: Think about dates on the x-axis and values on the y-axis.

Answer: Convert the date column to a datetime type, sort by date, use dates on the x-axis and the measured value on the y-axis, then format the date labels if necessary. A line chart is commonly used to show changes over time.

Example:

```
import pandas as pd
import matplotlib.pyplot as plt

df['Date'] = pd.to_datetime(df['Date'])
df = df.sort_values('Date')
plt.plot(df['Date'], df['Sales'])
plt.xlabel('Date')
plt.ylabel('Sales')
plt.show()
```

36. Write code to create an area chart showing monthly sales data.

Hint: Use plt.fill_between().

Answer: Plot the values and fill the area between the baseline and the sales values.

Example:

```
plt.plot(months, sales)
plt.fill_between(months, sales, alpha=0.3)
plt.title('Monthly Sales Area Chart')
plt.show()
```

37. How do you add annotations (such as arrows or text boxes) to highlight specific points in a chart?

Hint: There is a function specifically for annotations.

Answer: Use plt.annotate() or Axes.annotate(). It lets you place text and optionally draw an arrow from the text to a data point.

Example:

```
plt.annotate('Highest', xy=(x_max, y_max), xytext=(x_max, y_max + 20),
             arrowprops=dict(arrowstyle='->'))
```

38. How can you position the legend in a Matplotlib chart?

Hint: Think about the loc argument.

Answer: Use plt.legend(loc=...). Common locations include 'best', 'upper right', 'upper left', 'lower right', and 'lower left'. bbox_to_anchor can provide more precise placement.

Example:

```
plt.plot(x, y, label='Sales')
plt.legend(loc='upper left')
```

39. Write code to annotate the highest sales month in a line chart with an arrow.

Hint: Find the maximum sales value and its index, then use annotate().

Answer: Find the index of the maximum value, plot the series, and annotate that point.

Example:

```
import numpy as np

months = ['Jan', 'Feb', 'Mar', 'Apr']
sales = [120, 180, 150, 220]

plt.plot(months, sales, marker='o')
i = np.argmax(sales)
plt.annotate('Highest Sales', xy=(months[i], sales[i]),
             xytext=(months[i], sales[i] + 20),
             arrowprops=dict(arrowstyle='->'))
plt.show()
```

40. What function is used to save a Matplotlib plot to a file, and how do you specify the file format?

Hint: Think about plt.savefig().

Answer: Use plt.savefig(). The file format can be inferred from the filename extension, or specified explicitly with the format parameter.

Example:

```
plt.savefig('sales_chart.png', dpi=300, format='png', bbox_inches='tight')
```

41. How can you control the resolution (DPI) of a saved plot in Matplotlib?

Hint: Look at the savefig() arguments.

Answer: Use the dpi parameter in savefig(). A higher DPI generally produces a higher-resolution image but can increase file size.

Example:

```
plt.savefig('chart.png', dpi=300)
```

42. Describe how you would export a chart from Matplotlib to embed it in a PowerPoint presentation.

Hint: Think about saving an image with suitable resolution and dimensions.

Answer: Create and format the chart, then save it with plt.savefig(), typically as PNG for general use or SVG when editable/vector graphics are preferred and the workflow supports it. Choose an appropriate DPI and figure size, then insert the exported file into PowerPoint.

43. Given a retail sales dataset, which types of charts would you use to show KPIs like total sales, sales by category, and customer age distribution?

Hint: Think about trend, category, and distribution visuals.

Answer: For total sales, a KPI card or a simple highlighted number is appropriate. For sales by category, use a bar chart for clear comparison. For customer age distribution, use a histogram. If the dataset includes dates, a line chart can show sales trends over time.

44. Explain how you would integrate annotations and export options when preparing a visual analytics report using Matplotlib.

Hint: Combine clear storytelling annotations with high-quality export settings.

Answer: First identify the important findings, then use annotations to call attention to peaks, unusual values, targets, or other relevant points. Use clear titles, labels, and legends. Finally, export with savefig(), selecting an appropriate format, figure size, DPI, and bbox_inches='tight' so the chart remains readable in the final report.

45. How could you use an AI tool like ChatGPT or Copilot to help you write or debug Matplotlib code, and what steps would you take to verify the code works as expected?

Hint: Think about code suggestions and checking output or documentation.

Answer: AI can help generate starter code, explain Matplotlib functions, identify likely errors, suggest alternatives, and improve chart formatting. However, verify the result by running the code with your actual data, checking the output visually, testing edge cases, reading error messages, and consulting official documentation when behavior is unclear. Do not assume AI-generated code is automatically correct.

46. In what ways could generative AI tools speed up your data visualization workflow, and where would you still rely on your own skills and judgment?

Hint: Separate productivity benefits from analytical responsibility.

Answer: Generative AI can speed up repetitive coding, chart templates, debugging, documentation, and exploration of visualization ideas. Your own judgment is still essential for choosing the right chart, understanding the data, validating calculations, detecting misleading visualizations, protecting sensitive information, and explaining the business meaning of the results. AI should assist analysis, not replace analytical judgment.

Interview tip: For practical questions, explain what the function does, when you would use it, and give a short example. For chart-selection questions, always connect the chart type to the data type and analytical purpose.